PAP 2 – Python-Auswertung

Wichtige Funktionen für das PAP.

Dieses Dokument ist von mir (FEZ) selbstständig geschreiben und verfasst worden. Viele Funktionen sind von anderen übernommen und abgewandelt worden. Da der Code in VS Code geschrieben wird, wird teilweise auf den MS Copilot (K.I. Modell) zurückgegriffen. Dieser wird unterstützend und nicht generierend benutzt. Ich entschuldige mit hiermit auch formal für das unglaublich schlimme Denglisch in den Kommentaren und der Dokumentation.

Vorlage zum bestimmen aller im Praktikum benötigten Formel und intigriertem Latex Export für eine effizientere Nutzung.

Diese Version ist noch nicht aktiell. Daher führen viele Überschriften in Leere Kapitel. (Stand 07.03.2026)

Inhaltsverzeichnis für den Papulator

- Definition der Versuchsvariablen
- Import aller genutzen Libaries
- Funktionen der Fehlerrechnung
  - Runden signifikanter Stellen
  - Gaussische Fehlerfortpflanzung
  - Berechnung von Messwert und Fehler
  - Signifikante Standartabweichung zweier Messwerte
- Messdatenimporte
  - Patterns
  - Import von Experimentdaten
- Latex Export
  - Export von Gleichungen
  - Export von Einzelergebnissen
  - Export von Messreihen (Tabelle))
- Calc_Everything

- Plotting
- Ergebnisse der Aufgaben

Definition der Versuchsvariablen

Im Folgenden sind Versuchsvariablen definiert, die für einen besseren Workflow sorgen sollen. Diese werden zum exportieren und überschreiben von Dateien wichtig sein und ermöglichen es, diese Datei für jeden Versuch zu benutzen und lediglich die Variablen zu verändern. Es ist jedoch empfohlen eine Kopie der Forlage für jeden Versuch zu machen, damit dieses Dokument strukturiert bleibt.

Zudem sind wichtige Konstanten definiert, die immer wieder auftauchen.

In [3]:
```
versuchsnummer = "212"
versuchsname = "Beispielhafter-Name" # Auf Leerzeichen verzichten
aufgabe = "0z"

# Wichtige Konstanten
# Heidelberger Ortsgravitationsbeschleunigung in m/s^2
g_hd = 9.80984
err_g_hd = 0.00002

# Farben in hex
gruen="#1b722f"
hellgruen = "#1ba97f"
rot = "#b92d3a"
weinrot="#721c24"
blau="#073984"
lila = "#6610f2"
```

Import aller genutzen Libaries

In [4]:
```
# Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_HALF_UP, getcontext # Besonders für sig. Runden

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
```

```python
from scipy.special import factorial

import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path

import pandas as pd # Auch wichtig für den Latex Export
import csv
import re
from typing import List


# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML
```

Funktionen der Fehlerrechnung

Folglich werden eigene Funktionen definiert, die das Auswerten von Versuchen enorm vereinfachen sollen. Diese sind in vielen Punkten an fremden Code orientiert.

Runden signifikanter Stellen

In [5]:
```python
def round_sig_digs(errVal: float, val: float=None):
    """
    Function that rounds error value accoring the the standardised format:
    the function will search for the first digit that is greater than 2. If it found such a dig i
    It will than present the value according to the errVale, thus they have the same amount of di

    Parameter
    ---------
    **errValue** : float to be rounded to significant digit.

    **value** : float that will be rounded accoring to the errValue.

    Returnes
    --------
    **rounded** : Tupel
        Gibt den Fehler und den Messwert gemäß DIN gerundet zurück.
    """

    # ---------------------------------------------------------------------------
    #   Definition einer Hilfsfunktion, damit der Messwert auf die Stellen-
    #   länge des Fehlers angepasst werden kann und 9en keine Rundungs-
    #   probleme darstellen.
    # ---------------------------------------------------------------------------
    def round_with_carry(number_str):
```

```
        number_list = list(number_str)

        i = len(number_list) - 1

        # Solange 9 auf 0 setzen und nach links gehen
        while i >= 0 and int(number_list[i]) > 4:
            number_list[i] = '0'
            i -= 1

        # Wenn wir noch links eine Stelle haben erhöhen
        if i >= 0:
            number_list[i] = str(int(number_list[i]) + 1)
        else:
            # Alle Stellen waren 9 neue 1 vorne
            number_list.insert(0, '1')

        return ''.join(number_list)

    errVal = Decimal(str(errVal))
    val = Decimal(str(val))
    err_str = f"{errVal.normalize():f}"
    val_str = f"{val.normalize():f}"

    if '.' in err_str:
        err_int, err_frac = err_str.split('.')
    else:
        err_int = err_str
        err_frac = ''

    dez_pos = len(err_int) # Position des Kommas
    err_comb = err_int + err_frac + '0' # Alle Ziffern des Fehlers ohne Dezimal-trennung puls ext
    err_diffrent_dig = len(set(list(err_comb))) # Gibt die Anzahl der verschiedenn Ziffern an

    if '.' in val_str:
        val_int, val_frac = val_str.split('.')
    else:
        val_int = val_str
        val_frac = ''

    val_comb = val_int + val_frac


    leadZeros = False
    sig_dig_bool = False
    found_non_sig_dig = False

    fracZeros = 0
    non_sig_dig = ""

    rounded_err_dig = ""
    sig_dig_nine = False

    pos_non_sig_dig = 0

    val_round_up = False


    # Checken, ob Fehler kleiner ist als Messwert
    if errVal > val:
        print("Der Fehler ist größer als der Messwert")
        print("---------------- \n")
        return
```

```python
# ---------------------------------------------------------------------------
#   Untesucht ob und wo die Signifikante ist.
# ---------------------------------------------------------------------------

if err_int == '0':
    leadZeros = True

for numPos, dig in enumerate(err_comb):
    # Schaut ob dig sig ist
    if int(dig) > 2:
        # print(f"{dig} ist Signifikant")
        pos_sig_dig = numPos
        sig_dig_bool = True
        if dig == '9':
            sig_dig_nine = True
        break
    # Schaut ob dig nach dem Komma 0 ist
    elif (int(dig) == 0) and found_non_sig_dig and leadZeros:
        fracZeros = fracZeros + 1
        non_sig_dig = non_sig_dig + dig
    else:
        # print(f"{dig} ist nicht Signifikant")
        pos_non_sig_dig = numPos
        found_non_sig_dig = True
        non_sig_dig = non_sig_dig + dig
        # Schaut ob dig nicht signifikant ist. Dann soll
        if (len(non_sig_dig) - fracZeros) == 2:
            sig_dig_bool = False
            break
        # print(len(non_sig_dig.replace('0','')))
        pass


if leadZeros == True and sig_dig_bool == False:
    if (('1' in err_comb or '2' in err_comb)) and (err_diffrent_dig== 1) or (('1' in err_comb
        # print("Alle Ziffern sind nicht signifikant")
        pass

    elif (('0') in list(err_comb)) and (len(set(list(err_comb)))== 1):
        print("Die Ungenauigkeit ist null.")
        pass

    else:
        # print("Alle stellen sind 0, 1 oder 2 ")
        pass

# ---------------------------------------------------------------------------
#   Untersucht die Folgestelle der Signifikanten Stell: Auf- oder Abrunden?
# ---------------------------------------------------------------------------

# Korrekt runden
if sig_dig_bool == True:
    if (int(err_comb[pos_sig_dig + 1]) < 5):
        rounded_err_dig = int(dig)
        rounded_up = False
    else:
        rounded_err_dig = int(dig)  + 1
        rounded_up = True
```

```python
# 1 oder 2 und nach dem Komma
if sig_dig_bool == False and leadZeros and len(err_frac) > 1:
    if err_comb[pos_non_sig_dig + 1] == '0':
        rounded_err_dig = int(err_comb[pos_non_sig_dig + 1])
        rounded_up = False
    elif (int(err_comb[pos_non_sig_dig + 2]) < 5):
        # print(f"{err_comb[pos_non_sig_dig + 1]} muss abgerundet werden")
        rounded_err_dig = int(err_comb[pos_non_sig_dig + 1])
        rounded_up = False
    else:
        # print(f"{err_comb[pos_non_sig_dig + 1]} muss aufgerundet werden")
        rounded_err_dig = int(err_comb[pos_non_sig_dig + 1])  + 1
        rounded_up = True

    # Wissenschaftliche Schreibweise mit e fixen
    if 'e' in str(errVal):
        res = (str(errVal))
    else:
        rounded_err = non_sig_dig
        a = list(rounded_err)
        a.insert(dez_pos, '.')
        res = (''.join(a))

elif sig_dig_bool == False and leadZeros and len(err_frac) == 1:
   rounded_err_dig = err_frac + '0'
   rounded_up = False


# 1 oder 2 vor dem Komma
if (sig_dig_bool == False) and (leadZeros == False):
    if (int(err_comb[pos_non_sig_dig + 1]) < 5):
        # print(f"{err_comb[pos_non_sig_dig]} muss abgerundet werden")
        rounded_err_dig = int(err_comb[pos_non_sig_dig])
        rounded_up = False
    else:
        # print(f"{err_comb[pos_non_sig_dig]} muss aufgerundet werden")
        rounded_err_dig = int(err_comb[pos_non_sig_dig])  + 1
        rounded_up = True

    # Wissenschaftliche Schreibweise mit e fixen
    if 'e' in str(errVal):
        res = (str(errVal))
    else:
        rounded_err = non_sig_dig
        a = list(rounded_err)
        a.insert(dez_pos, '.')
        res = (''.join(a))

# Neunen richtig runden
elif sig_dig_nine and rounded_up and sig_dig_bool == False:
    rounded_err = round_with_carry(err_int)
    print(rounded_err)
    res = rounded_err

# Nullen ordentlich betrachten
elif leadZeros and err_int == "0" and sig_dig_bool:
    rounded_err = non_sig_dig + str(rounded_err_dig)
    a = list(rounded_err)
    a.insert(dez_pos, '.')
    res = ''.join(a)

# Richtig runden, wenn Stelle vor dem Komma leigt
```

```
    elif dig in err_int:
        rounded_err = non_sig_dig + str(rounded_err_dig) + (len(err_int) - pos_sig_dig - 1) * '0'
        res = rounded_err
    else:
        rounded_err = non_sig_dig + str(rounded_err_dig)
        a = list(rounded_err)
        a.insert(dez_pos, '.')
        res = ''.join(a)


    if err_comb[pos_non_sig_dig + 1] == '9':
        sig_dig_nine = True

    if sig_dig_nine and rounded_up and leadZeros:
        if sig_dig_nine and rounded_up:
            rounded_err = non_sig_dig[:len(non_sig_dig) - 1] + str(int(dig) + 1)
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)
        elif rounded_up:
            rounded_err = non_sig_dig[:len(non_sig_dig)] + str(int(err_comb[len(non_sig_dig)]) +
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)
        else:
            rounded_err = non_sig_dig[:len(non_sig_dig)] + err_comb[len(non_sig_dig)]
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)

    # Runden, wenn die Stelle nicht signifikant ist
    # Nach dem Komma
    elif leadZeros and sig_dig_bool == False:
        if sig_dig_nine and rounded_up:
            rounded_err = non_sig_dig[:len(non_sig_dig) - 1] + str(int(dig) + 1) + '0'
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)
        elif rounded_up:
            rounded_err = non_sig_dig[:len(non_sig_dig)] + str(int(err_comb[len(non_sig_dig)]) +
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)
        else:
            rounded_err = non_sig_dig[:len(non_sig_dig)] + err_comb[len(non_sig_dig)]
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)

    # Vor dem Komma
    if (leadZeros == False) and (sig_dig_bool == False):
        if rounded_up:
            rounded_err = err_int[0] + str(rounded_err_dig) + '0' * (len(err_int) - 2)
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)
        # elif (rounded_up == False) and (len(err_int) == 2):
        #     rounded_err = non_sig_dig[:len(non_sig_dig)]
        #     a = list(rounded_err)
        #     a.insert(dez_pos, '.')
        #     res = ''.join(a)
        else:
```

```
            rounded_err = non_sig_dig[:len(non_sig_dig)] + '0' * (len(err_int) - 2)
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)

        if int(val_int[abs(len(err_int) - len(val_int)) + 1]) < 5:
            val_round_up = False
        else:
            val_round_up = True


    # ---------------------------------------------------------------------------
    #   Anpassen des Messwertes an den gerundeten Fehler (Gleich gerundet)
    # ---------------------------------------------------------------------------

    last_zero = False
    if res[len(str(res)) - 1] == "0":
        last_zero = True

    val = Decimal(str(val))
    res = Decimal(str(res))

    # Exponent (z.B. -2 bei 0.12, +2 bei 200)
    num_decimals = -res.normalize().as_tuple().exponent

    # Rundungsstelle bestimmen
    if (leadZeros == False):
        if val_round_up:
            rounded_val = val_int[:abs(len(err_int) -  len(val_int))] + str(int(val_int[abs(len(e
        else:
            rounded_val = val_int[:abs(len(err_int) -  len(val_int))] + str(int(val_int[abs(len(e
    elif sig_dig_bool:
        rounded_val = val.quantize(Decimal('1.' + '0' * (num_decimals)), rounding=ROUND_HALF_UP)
    elif last_zero:
        rounded_val = Decimal(str(val.quantize(Decimal('1.' + '0' * (num_decimals)), rounding=ROU
    else:
        rounded_val = Decimal(str(val.quantize(Decimal('1.' + '0' * (num_decimals)), rounding=ROU

    # Debugging Print
    print(f"Aus den Messwert {val} mit der Ungenauigkeit {errVal} berechnet sich das Ergebnis zu:
    print("---------------- \n")

    return rounded_val, res
```

Gaussische Fehlerfortpflanzung

In [6]:
```
def gff(func, errPronePar):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    ----------
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind ungleich den Werten f
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc. genutzt und für deren F
```

```
    Return
    ----------
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParamters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParamters = []
    for errPar in errPronePar:
        d = sp.symbols('thisWillTurnDelta' + errPar.name)
        partial = sp.diff(func, errPar) * d  # Die Funktion wird nach der fehlerbehafteten Variab
        error = error + partial**2 # Fehler werden quadratisch aufsummiert
        errProneParamters.append((errPar,d))
    absolut_err=sp.simplify(sp.sqrt(error),rational = True)
    relativ_err=sp.simplify(sp.sqrt(error/func**2),rational = True)

    # Debugging Print
    # print(f"Die Fehlerfunktion zur Ausgangsfunktion {func} wurde erfolgreich nach den Fehlergrö

    return absolut_err, relativ_err, errProneParamters
```

Berechnung von Messwert und Fehler

In [7]:
```
def calc_with_err(func, errFunc, values):
    """
    Methode zum berechnen von Werten und deren Fehler.

    Parameter
    ----------
    **func** : sympy function
         Funktionen mit Parametern.

    **errFunc** : sympy function
        Die zu func gehörende Fehlerformel. Wird über gff bestimmt.

    **values** :
        Werte, die in die Funktionen eingesetzt werden.
        Als array von Tupeln der Form [(a,da),(b,db),...] oder als array/liste [a,da,b,db,...]
        (Reihenfolge muss die sein, in der die Argumente in der Funktion genommen werden)

    Return
    ----------

    """

    #Falls der Input in mehrere Tupel aufgeteilt ist, werden diese zu einem Array zusammengefügt
    if (np.ndim(values) != 1):
        values = np.concatenate(values)
    result = func(*values[::2])
    uncertainty = errFunc(*values)

    # Debigging Print
    # print(f"Die Ergebnisse wurden erfolgreich durch Funktion und Fehlerfunktion bestimmt!")
```

```
    print(f"{result} \\pm {uncertainty}")

    return result, uncertainty
```

Fehlerfunktion und Messwerte

Signifikante Standartabweichung zweier Messwerte

In [8]:
```
def sigma_abweichung(p1, p2, err_p1 = 0, err_p2 = 0):
    """
    Funktion zum berechnen der Sigma-Abweichugn von zwei Messwerten, oder einem Messwert und eine
    """
    if err_p1 == err_p2 == 0:
        print("Für die Sigma-Abweichung muss mindestens ein Wert fehlerbehaftet sein!")
    else:
        abweichung = abs(p1 - p2)/(np.sqrt(err_p1 ** 2, err_p2 ** 2))
    return abweichung
```

Messdatenimporte

Für die Auswertung von Versuchen ist es nicht selten, dass Messwerte in *CSV* oder in *TXT* Dateien gespeichert sind. Daher sollen diese Vernünftig eingelesen und verarbeitet werden, um das Benutzen der Daten möglichst einfach und instuinktiv zu gestalten.

Um das Handeln von CSV Tabellen möglichst einfach zu gestalten, werden RIGEX-Pattern genutzt. Das heißt, wenn die Tabelle vor einem Versuch ordentlich strukturiert ist, können die Werte sofort genutzt werden, so wie es gewünscht ist. Dabei sollen folgende Eigenschaften erfüllt sein:

- Das Nutzen einer Kopfzeile ist nicht nur möglich, sondern explizit empfohlen.
- Erkennung eines Messwertes, seiner Einheit und einer Potenz. Das hat dan bspw. die Form: *(name [unit] 10^{power})*
- Der zum Messwert gehörende Fehler. Dies ist meist ein Ablese Fehler. Die Bennenung sollte dabei: *err name* sein.
- Direkte Auswerten von Messreihen. Soll bspw. über mehrere Werte gemittelt werden, so wird automatisiert Mittelwert, Standartabweichung und gesamt Fehler der Messreihe bestimmt.
- Dimensionslose Größen sollen erkannt werden.
- Einstellung eines Indexes. Dies kann entweder die nummerische Angabe des Messdurchganges sein, oder der Parameter-Name. Wird bspw. dieselbe Messung in Wasser und in Luft durchgeführt, so sollen dies die Parameter-Namen sein. Diese sollen in der ersten Spalte der Tabelle stehen.

Patterns

In [9]:
```
# RegEx Patterns, um autoamtisiert Schemata anzuwenden und Daten anhand dieser auszuwerten.

# Finden eines Messwertes, seiner Einheit und seiner Potenz
pattern = re.compile(
        r'^\s*'
```

```
        r'(?P<name>[^\[]+?)'    # alles bis zur ersten '['      # Name
        r'\s*\[\s*'             # echtes '['
        r'(?P<unit>[^\]]+?)'    # alles bis zur ']'             # Einheit
        r'\s*\]'                # echtes ']'
        r'(?:\s*10\^(?P<power>-?\d+))?'                         # optionale 10^n
        r'\s*$'
    )

series_pattern = series_pattern = re.compile(
        r'^(?P<base>\w+)_\d+'
        r'(?:\s*\[\s*(?P<unit>[^\]]+)\s*\])?'
    )

# Index Spalte idx
index_pattern = re.compile(
    r'idx\s*'
    )

# Spalte die Ignoriert werden sill ign
ignore_pattern = re.compile(
    r'(?:ign[^,]*\s*,\s*)?'
)

# Messwert mit Einheit
messurment_pattern = re.compile(
    r'(?P<value>Messwert)\s*$$(?P<unit>[^$$]+)\]\s*,\s*'
)

# Fehler des Messwertes
error_pattern = re.compile(
    r'(?:err\s+Messwert\s*,\s*)?'
)
```

Import von Experimentdaten

Das einlesen von Messweerten und das speichern dieser als Arrays für einen besseren zugriff. Automatisches herausfinden von Messwert und dazugehörigem Fehler.

In [10]:
```
def import_experimental_data(path:str = versuchsnummer + ".csv", setDelimiter:str = ",", setHeade
    """

    Parameter
    ---------
    **path**: str
        Relatives verzeichnes der einzulesenden Datei. Am besten CSV-Datei einfach unter *versuch

    **setDelimiter**: str
        Default ist hier das Komma:  > , <

    **setHeader**: int
        Setzt fest, was die Header-Row ist. Default ist 0 (die oberste Row)

    Return
    ---------
    **cleanded** : Pandas DataFrame
        DataFrame, welches sicher zum weiterverwenden ist. NaN und ähnliches werden herausgefilte
    """

    # Liest die Daten unmaipuliert
```

```
    try:
        df = pd.read_csv(path, delimiter=setDelimiter, header=0, index_col=None)
    except FileNotFoundError:
        print(f"The file {path} was not found.")

    # Wir erwarten nicht, dass zwei perfekt identische Zeilen existieren koennten, daher werden d
    # Langfristig soll das noch verschönert werden und wirklich nur NaN Zeilen gelöscht werden
    no_NaN_data = df.drop_duplicates(keep= False)
    df.reset_index(drop=True, inplace=True)

    return no_NaN_data

# Speichert direkt das DataFrame als Variable, so kann dieses direkt in anderen Cells benutzt wer
experimental_data = import_experimental_data()
```

Latex Export

Erstellen der dedizierten Dartei zum Speichern der Werte für **LATEXT**

Folgender Code sollte einmal ausgeführt werden, damit direkt die Datei zum Speichern erstellt werden kann. Unbedingt daran denken, die Versuchsnummer anzupassen, die Werte anderer Versuche könnten ansonsten verloren gehen. Es gibt jedoch eine sicherheits Kopie.

Die File wird Folgendes beinhalten:

- Formel, sowie Formel nach gff
- Berechnete Werte und deren Fehler
  - Wert + Fehler
  - Tabellen Export

In [11]:
```
def create_tex_result_file(fileName:str = f"python-results.sty", relativePath:str="../Auswertung/
    """
    Diese Method erstellt automatisch eine tex-Datei, in dem Messwerte bzw. deren Ergebnisse, Tab
    Die File wird unter *Versuche/${versuchsnummer}$/Auswertung/python-results.tex* zufinden seir
    angepasst werden muss.

    Die Python-File muss im Python-Ordner des jeweiligen Versuches liegen!

    Parameter
    ----------
    **fileName** :
        neuer Name, falls die Datei besonders heißen soll.

    Return
    --------
    **path** : str
        Gibt den Path zur file wieder, welche autoamtisch generiert. In diese sollen später die F
    """

    path = relativePath + fileName

    if os.path.isfile(path):
        print(f"{fileName} existiert bereits.")
```

```
        pass
    else:
        print(f"{fileName} wird in {relativePath} erzeugt.")
        with open(path, 'w') as file:
            file.write("% Dies ist eine automatisch generierte Datei. Hier werden automatisiert \

        with open(path + "-BackUp.sty", 'w') as bFile:
            bFile.write("% Dies ist eine automatisch generierte Datei. Hier wird dediziert ein Ba

    return path

# pyPath ist die File, in die der Latex Export geschehen soll.
pyPath = create_tex_result_file()
```

```
python-results.sty existiert bereits.
```

Export von Gleichungen

In [12]:

```
def function_to_latex(func, texVarName:str=None, texCom:str=None):
    """
    Zeigt die Formel als gerenderte Math-Darstellung und darunter
    den Latex-Quelltext, der per Button kopiert werden kann. (Für leichtere Benutzung als HTML).

    Zudem wird die Latexformel als Variable in Latex gespeichert.

    Parameters
    ----------
    **f** : sympy function

    **texVarName** : str
        Einzigartiger Name der Variablen. Dieser wird zum überschreiben alter Formel gebraucht.

    **texCom** : str
        setzt den newcommand-Kürzel für latex fest. Setzte kein Backslash! Auch texCom muss einzi
    """
    print("Die gegebene Funktion lautet: \n")
    display(Math(sp.latex(func, long_frac_ratio=2).replace('thisWillTurnDelta', r'\Delta '))

    print("Hier ist der dazugehörige Latex code: \n")
    latex_str = sp.latex(func, long_frac_ratio=2).replace('thisWillTurnDelta', r'\Delta ')
    html = f"""
    <div style="margin-top:0.5em;">
        <code id="latex-code-{id(func)}">
            {latex_str}
        </code>
        <button onclick="
            const tex_as_txt = document.getElementById('latex-code-{id(func)}').innerText;
            navigator.clipboard.writeText(tex_as_txt)
        " style="
            margin-left:8px;
            padding:2px 6px;
            cursor:pointer;
        ">
            Kopieren
        </button>
    </div>
    """
    display(HTML(html))


# ---------------------------------------------
```

```
#   Diese Funktion hat leider einen Bug.
# -------------------------------------------

    # # Hinzufuegen bzw. Ueberschreieben der Formel in die Sammlung
    # with open(pyPath, 'r') as file:
    #     lines = file.readlines()
    # found = False

    # with open(pyPath, 'r') as file:
    #     for lineNum,  line in enumerate(lines, 1):
    #         if texVarName in line: # Checkt, ob der Variablen Name bereits vergeben ist.
    #             lines[lineNum] = "\\newcommand{\\" + texCom + "}{" + latex_str + "} \n\n"
    #             found = True
    #             # Deugging print
    #             # print(f'{texVarName} is at line {lineNum}')

    # if not found:
    #     print("Die neue Funktion wurde hinzugefügt")
    #     with open(pyPath, 'w') as file:
    #         file.writelines(lines) # Schreibt den alten Stand
    #         file.write("\n\n% " + texVarName + "\n") # Fügt die Variable als Kommentar hinzu
    #         file.write("\\newcommand{\\" + texCom + "}{" + latex_str + "} \n\n")
    # else:
    #     print("Die alte Funktion wurde erfolgreich überschrieben.")
    #     with open(pyPath, 'w') as file:
    #         file.writelines(lines)
```

Export von Einzelergebnissen

Export von Messreihen (Tabelle)

In [13]:
```
def table_to_latex(df_tex: str, texVarName: str, texCom: str):
    """
    Fügt aus einem DataFrame entstandene Latex Tabelle der special Python File hinzu.

    Parameters
    ----------
    **df_tex** : pd.DataFrame.to_latex
        Uebersetztes DataFrame. Kann Label und Caption handlen. Dezimaltrennung steht auf ','.

    **texVarName** : str
        Einzigartiger Name der Variablen. Dieser wird zum überschreiben alter Formel gebraucht.

    **texCom** : str
        setzt den newcommand-Kürzel für latex fest. Setzte kein Backslash! Auch texCom muss einzi

    Returns
    -------
    Schreibt automatisch in die sty file
    """

    # Hinzufuegen bzw. Ueberschreieben der Tabelle in die Sammlung
    with open(pyPath, 'r') as file:
        lines: List[str] = file.readlines()
    found = False

    with open(pyPath, 'r') as file:
        for lineNum,  line in enumerate(lines):
```

```python
            if texVarName in line: # Checkt, ob der Variablen Name bereits vergeben ist.
                print(f'{texVarName} starts at line {lineNum + 1}')
                found = True
                table_start_line = lineNum
                break

    if not found:
        print(f"Die neue Tabelle ({texVarName}) wurde hinzugefügt")
        with open(pyPath, 'w') as file:
            file.writelines(lines) # Schreibt den alten Stand
            file.write("\n% " + texVarName + "\n")
            file.write("\\newcommand{\\" + texCom + "}{" + df_tex + "}\n\n")
    else:
        print(f"Die alte Tabelle ({texVarName}) wird überschrieben.")
        start_table = table_start_line + 1
        while start_table < len(lines) and lines[start_table].strip() == "":
            # Leere Zeile überspringen – das ist unser Marker
            start_table += 1
            break   # Wir wollen nur die erste leere Zeile

        # Falls kein leerer Marker gefunden wurde, gehen wir davon aus,
        # dass die Tabelle direkt in der nächsten Zeile startet.
        if start_table >= len(lines):
            start_table = table_start_line + 1

        end_table = start_table

        while end_table < len(lines):
            cur = lines[end_table].strip()

            # Abbruchbedingungen:
            if (
                cur.startswith(r"\\newcommand")
                or cur.startswith(r"\\section")
                or cur.startswith(r"\\subsection")
                or cur.startswith(r"\\begin")
            ):
                break

            end_table += 1

        if end_table > start_table:
            del lines[start_table - 1:end_table]

        insertion_point = table_start_line + 1  # Direkt nach der Zeile mit texVarName
        new_block = [
            # "\n\n",
            f"% {texVarName}\n",
            f"\\newcommand{{\\{texCom}}}{{{df_tex}}}\n",
        ]
        lines[insertion_point:insertion_point] = new_block

        with open(pyPath, 'w') as file:
            file.writelines(lines)
```

Finaler Export

Für die Auswertung sollen Messwerte und Fehler in einer Zelle stehen. Dies tut *tab_merge_val_err* automatisiert.

In [14]:
```python
def tab_merge_val_err(df: pd.DataFrame, show_series_values:bool=False, setTexTabCap: str="", setT
    """Sucht im DataFrame nach einem Asdruck:
        "<name> [<unit>] <power>"
    und dem dazugehörigem Fehlerausdruck:
        "err <name>".
    Erstellt ein neues DataFrame für den Latex-Export

    Parameters
    ----------
    **df* : pd.DataFrame
        Original data frame.

    **show_series_values** : bool
        Rather the table should show the experimental data or just its mean


    Returns
    -------
    pd.DataFrame
        Eine Kopie von ``df``.
    """

    # Kopiert das alte DataFrame
    df_tex = pd.DataFrame(index=df.index)


    def format_tex(row):
        val = row[col]
        err = row[err_col]
        # return f'({val} \\pm {err}) \\mathrm{{{header_unit}}}'
        val_r, err_r = round_sig_digs(err, val)
        return f'${val_r} \\pm {err_r}$'

    # ----------------------------
    # 1) Einzelmessungen
    # ----------------------------

    for col in df.columns:
        col_match = pattern.match(col)

        if not col_match:
            # No header that follows the expected schema – skip it
            # print(f'Skipping column {col!r}: not "<value> [<unit>] <power>"')
            continue

        # Extract the captured parts
        name = col_match.group('name').strip()
        unit = col_match.group('unit').strip()
        power = col_match.group("power")
        # print(f'Found measurement column: name={name!r}, unit={unit!r}, power={power!r}')

        # Build the name of the error column that must exist
        err_col = f'err {name}'
        if err_col not in df.columns:
            # print(f'No error column "{err_col}" for measurement "{name}" - skipping')
            continue

        if power:
            header_unit = f"$\\mathrm{{{unit}}} \\cdot 10^{{{power}}}$"
        else:
            header_unit = f"$\\mathrm{{{unit}}}$"
```

```python
    df_tex[f'{name} [{header_unit}]'] = df.apply(format_tex, axis=1)


# ----------------------------
# 2) Messreihen
# ----------------------------
grouped = {}

for col in df.columns:
    series_match = series_pattern.match(col)
    if series_match:
        base = series_match.group("base")
        unit = series_match.group("unit")
        # power = col_match.group("power")
        grouped.setdefault(base, {"cols": [], "unit": unit})
        grouped[base]["cols"].append(col)


# --- Berechnungen ---
for base, info in grouped.items():
    cols = info["cols"]
    series_unit = info["unit"] or ""


    err_col = f"err {base}"
    if err_col not in df.columns:
        continue

    values = df[cols]

    if show_series_values == True:
        # Einzelwerte in DataFrame hinzufuegen
        for ind, col in enumerate(cols, start=1):
            df_tex[f'{base}_{ind} [{unit}]'] = df.apply(format_tex, axis=1)

    # Mittelwert
    mean = values.mean(axis=1)

    # Standardabweichung
    std = values.std(axis=1, ddof=1)

    # Fehler des Mittelwerts
    stat_err = std / np.sqrt(len(cols))

    # systematischer Fehler
    sys_err = df[err_col]

    # Gesamtfehler
    total_err = np.sqrt(stat_err**2 + sys_err**2)

    # if power:
    #     header_unit = f"\\mathrm{{{unit}}} \\cdot 10^{{{power}}}"
    # else:
    #     header_unit = f"\\mathrm{{{unit}}}"

    formatted = []

    # for m, e in zip(mean, total_err):
    #     m_r, e_r = round_sig_digs(e, m)
    #     formatted.append(f"${m_r} \\pm {e_r}$")
```

```
        df_tex[f"$\\overline{{{base}}} [\\mathrm{{{series_unit}}}]$"] = formatted

    setTexTabCap="Dies ist die Caption der Tabelle"
    setTexTabLab="bspTab"

    table_to_latex(df_tex.to_latex(decimal=',', caption=setTexTabCap, label=setTexTabLab, positic

    print(df_tex.to_latex(decimal=',', caption=setTexTabCap, label=setTexTabLab, position="h"))
    return df_tex.to_latex(decimal=',', caption=setTexTabCap, label=setTexTabLab, position="h")


# combine_value_error(clean_data)
```

Berechnen neuer Spalten mit Fehler

Die folgende Funktion berechnet anhand einer gegebenen Formel und den gegebenen Parametern die Ergebnisse einer neuen Spalte mit Ihren Fehlern.

Calc_everything

In [15]:
```
def calc_everything(function, params, data, params_without_error=[], print_formula:bool=True):
    """
    Returned Berechneten Wert plus Fehler. Zudem werden die Formeln als Latex-Code ausgeworfen.

    Parameter
    ----------
    **function**: sympy Funktion mit Parametern. In diese werden die Messwerte eingesetzt.

    **params**: Parameter der Funktion. Diese werden als Array von Sympy-Symbolen gebraucht. Bspw

    **data**: 2D-Array mit den Messdaten, sodass die Zeilen die Form haben: [Parameter 1, Fehler
        Die Funktion wird zeilenweise angewandt. Wird kein Fehler für einen Parameter angenommen,
        an die Funktion gegeben werden oder ganz weggelassen werden. Dann muss allerdings der bet

    **params_without_error**: Alle Parameter zu denen kein Fehler explizit in den Daten angegeber
        auch nicht in der Latex Form der Fehlerformel vor

    """

    exp_data = np.zeros((data.shape[0],data.shape[1]+len(params_without_error)))
    i = 0      # läuft durch die Parameter
    j = 0      # läuft durch die expanded data
    z = 0      # läuft durch die eingegebene data, also die Messwerte und deren Fehler
    # Läuft durch jeden Parameter und seinen Fehler
    while (i < len(params)):
        # Checkt, ob der Parameter Fehlerbehaftet ist. Wenn, dann wird an j-ter Stelle des exp_da
        if (params[i] in params_without_error):
            exp_data[:,j] = data[:,z]
            i = i + 1
            j = j + 2
            z = z + 1
        else:
            exp_data[:,j] = data[:,z]
            exp_data[:,j+1] = data[:,z+1]
            i = i + 1
```

```
            j = j + 2
            z = z + 2
    # print(exp_data)

    # Create variable that stores parameters that have no assigned uncertainty
    params_with_error = []
    j = 0
    for n in np.arange(0,len(params)):
        if not (params[n] in params_without_error):
            params_with_error.append(params[n])
            j = j + 1

    # Get the given function and error function as numpy functions
    f = sp.lambdify(params, function, "numpy")
    absolut_err, relativ_err, parameters = gff(function,params) # Gauss Fehlerfortpflanzung
    err_abs = sp.lambdify(np.concatenate(parameters), absolut_err, "numpy")

    # f = sp.lambdify(params,function)
    # err_abs = sp.lambdify(np.concatenate(parameters),absolut_err)

    # Calculate the results for each row of data
    results = np.zeros((data.shape[0],2))
    for n in np.arange(0,data.shape[0]):
        results[n,:] = calc_with_err(f, err_abs, exp_data[n,:])
        # print(calc_with_err(f, err_abs, exp_data[n]))

    # if (len(results) < 10):
    #     print("Results:")
    #     print(results)

    # Substitutes 0 for the uncertainty of the parameters without error, so it doesnt show up in
    for p in params_without_error:
        absolut_err = absolut_err.subs('thisWillTurnDelta'+p.name,0)
    for p in params_without_error:
        relativ_err = relativ_err.subs('thisWillTurnDelta'+p.name,0)

    # Wiedergabe des Latex-Codes
    function = sp.simplify(function,symbols = params, rational= True)
    function = sp.separatevars(function)

    if print_formula:
        print("gegebene Funktion:")
        function_to_latex(function)

        print("Formel des absoluten Fehlers der gegebenen Funktion:")
        function_to_latex(absolut_err)

        print("Formel des relativen Fehlers der gegebenen Funktion:")
        function_to_latex(relativ_err)

    return(results)
```

Plotting

Ergebnisse der Aufgaben

Aufgabe 1

Bestimmung der Viskosität nach Stokes mit einem Kugelfallviskosimeter

In [16]:
```
aufgabe = "1"
```

Definieren der Werte aus der eingelesenen CSV

In [80]:
```
df = experimental_data

# Radius Quadrat
rSqr_val = (df["r [mm]"])**2
rSqr_err = df["d [mm]"] * df["err d"]/2

# Durchschnittliche Zeit pro Messung
t_avg = []
t_err = df["err t"]
for idx, row in df.iterrows():
    # Beispiel: Ausgabe aller t-Werte dieser Zeile
    t_vals = row[['t_1 [s]', 't_2 [s]', 't_3 [s]', 't_4 [s]', 't_5 [s]']]
    # print(f"Messung {idx+1}: t-Werte = {t_vals.values}")
    avg = np.average(t_vals.values)
    t_avg.append(avg)
    # print(f"Durchscnittliche Zeit der Messung{idx+1}: {avg} \n -------")

s_val = df["s [mm]"]
s_err = df["err s"]

# Durchschnittliche Geschwindigkeit pro Messung
v_val = []
v_err = []
for idx, row in df.iterrows():
    val = "s [mm]"
    err = "err s"
    ges_val = df.iloc[idx:idx+1][val]/t_avg[idx]
    ges_err = np.sqrt((s_err[idx] / t_avg[idx])**2 + (s_val[idx] * t_err[idx] / (t_avg[idx])**2)*
    v_val.append(ges_val)
    v_err.append(ges_err)

# Lambda Korrektur
lam_val = []
lam_err = []
for idx, row in df.iterrows():
    val = "r [mm]"
    err = "err d"
    l_val = 1 + 2.1 * df.iloc[idx:idx+1][val]/(75)
    l_err = 2.1 * df.iloc[idx:idx+1][err]/(75)
    lam_val.append((l_val))
    lam_err.append((l_err))

# Korrigierte Geschwindigkeit v_s
vs_val = []
```

```
vs_err = []
for idx, row in df.iterrows():
    s_val = (v_val[idx] * lam_val[idx])
    s_err = np.sqrt((v_val[idx] * lam_err[idx])**2 + (v_err[idx] * lam_val[idx])**2)
    vs_val.append((s_val))
    vs_err.append(s_err)

print("Wert:")
print(np.array(vs_val))
print("Fehler:")
print(np.array(vs_err))
```

```
Wert:
[[ 0.64144737]
 [ 2.05764612]
 [ 4.73636364]
 [ 7.89631107]
 [12.44186047]
 [16.27627628]
 [22.54620123]
 [28.15189873]
 [34.75308642]]
Fehler:
[[0.02577695]
 [0.08594693]
 [0.11469452]
 [0.15829558]
 [0.33362719]
 [0.40122953]
 [0.73026621]
 [1.10564691]
 [1.64618644]]
```

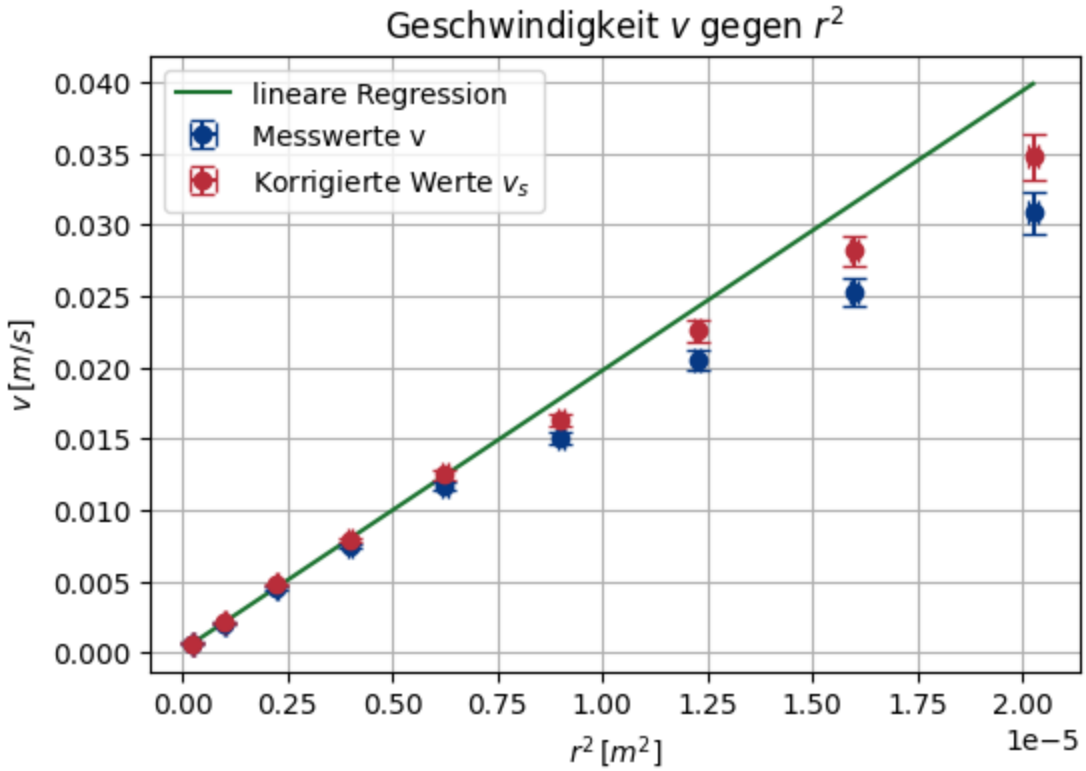
In [18]:

```
# Einheiten umrechnen

# r^2
x = rSqr_val.values * 1e-6
x_err = rSqr_err.values * 1e-6

# v
y = np.array([float(v) for v in v_val]) * 1e-3
y_err = np.array([float(v) for v in v_err]) * 1e-3

# vs
ys = np.array([float(vs) for vs in vs_val]) * 1e-3
ys_err = np.array([float(vs) for vs in vs_err]) * 1e-3

# Fehlergerade
x_fit = x[:5]
y_fit = ys[:5]

p, cov = np.polyfit(x_fit, y_fit, 1, cov=True)

k, b = p

k_err = np.sqrt(cov[0,0])
b_err = np.sqrt(cov[1,1])

print(f"Steigung k = ({k:.3e} \\pm {k_err:.1e})")
print(f"Achsenabschnitt b = ({b:.3e} \\pm {b_err:.1e})")

x_line = np.linspace(min(x), max(x), 100)
```

```python
y_line = k * x_line + b

# plot
plt.figure(figsize=(6,4))
plt.plot(x_line, y_line, color=gruen, label="lineare Regression")

plt.errorbar(
    x, y,
    xerr=x_err,
    yerr=y_err,
    fmt='o',
    capsize=4,
    color=blau,
    label=r"Messwerte v"
)

plt.errorbar(
    x, ys,
    xerr=x_err,
    yerr=ys_err,
    fmt='o',
    capsize=4,
    color=rot,
    label=r"Korrigierte Werte $v_s$"
)

plt.xlabel(r"$r^2\,[m^2]$")
plt.ylabel(r"$v\,[m/s]$")
plt.title("Geschwindigkeit $v$ gegen $r^2$")
plt.grid(True)
plt.legend()

plt.savefig("../img/plot/ges_gegen_r2.pdf", dpi=300)
plt.show()
```

```
Steigung k = (1.961e+03 \pm 2.5e+01)
Achsenabschnitt b = (1.613e-04 \pm 8.7e-05)
```

```
C:\Users\finnz\AppData\Local\Temp\ipykernel_5428\152146680.py:8: FutureWarning: Calling float on
a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.ilo
c[0]) instead
  y = np.array([float(v) for v in v_val]) * 1e-3
C:\Users\finnz\AppData\Local\Temp\ipykernel_5428\152146680.py:12: FutureWarning: Calling float on
a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.ilo
c[0]) instead
  ys = np.array([float(vs) for vs in vs_val]) * 1e-3
C:\Users\finnz\AppData\Local\Temp\ipykernel_5428\152146680.py:13: FutureWarning: Calling float on
a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.ilo
c[0]) instead
  ys_err = np.array([float(vs) for vs in vs_err]) * 1e-3
```



Steigung bestimmen:

In [19]:
```
vs_sym, r_sym = sp.symbols('v, r')
params = [vs_sym, r_sym]

k = vs_sym / r_sym**2

abs_err, rel_err, pa = gff(k, params)
function_to_latex(abs_err, texVarName="Steigungs Fehler", texCom="k_err")

# values = []
# calc_with_err(k, abs_err, values)
```

```
Die gegebene Funktion lautet:
```

$$\sqrt{\frac{1}{r^6}\left(r^2\Delta v^2 + 4\Delta r^2 v^2\right)}$$

```
Hier ist der dazugehörige Latex code:
```

```
\sqrt{\frac{1}{r^{6}} \left(r^{2} \Delta v^{2} + 4 \Delta r^{2} v^{2}\right)}
```

Kopieren

Bestimmung der Viskosität

In [20]:
```
g_sym, pk_sym, pf_sym, k_sym = sp.symbols('g, pk, pf, k')
params=[g_sym, pk_sym, pf_sym, k_sym]

eta = 2/9 * g_sym * (pk_sym - pf_sym) / k_sym

values = np.array([[g_hd, err_g_hd, 1385, 15,1148.2, 1.1, 1961, 25]])

calc_everything(eta, params, values, [])
```

```
round_sig_digs(0.017053146577525387, 0.2632409895178194)
```

```
0.2632409895178194 \pm 0.017053146577525387
gegebene Funktion:
Die gegebene Funktion lautet:
```

$$\frac{2g}{9k}(-pf + pk)$$

```
Hier ist der dazugehörige Latex code:
```

```
\frac{2 g}{9 k} \left(- pf + pk\right)
```

Kopieren

```
Formel des absoluten Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:
```

$$\frac{2}{9}\sqrt{\frac{1}{k^4}\left(g^2\Delta k^2(pf - pk)^2 + k^2\left(g^2\Delta pf^2 + g^2\Delta pk^2 + \Delta g^2(pf - pk)^2\right)\right)}$$

```
Hier ist der dazugehörige Latex code:
```

```
\frac{2}{9} \sqrt{\frac{1}{k^{4}} \left(g^{2} \Delta k^{2} \left(pf - pk\right)^{2} + k^{2} \left(g^{2} \Delta pf^{2} + g^{2} \Delta pk^{2} + \Delta g^{2} \left(pf - pk\right)^{2}\right)\right)}
```

Kopieren

```
Formel des relativen Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:
```

$$\sqrt{\frac{\Delta pf^2}{(pf - pk)^2} + \frac{\Delta pk^2}{(pf - pk)^2} + \frac{\Delta k^2}{k^2} + \frac{\Delta g^2}{g^2}}$$

```
Hier ist der dazugehörige Latex code:
```

```
\sqrt{\frac{\Delta pf^{2}}{\left(pf - pk\right)^{2}} + \frac{\Delta pk^{2}}{\left(pf - pk\right)^{2}} + \frac{\Delta k^{2}}{k^{2}} + \frac{\Delta g^{2}}{g^{2}}}
```

Kopieren

```
Aus den Messwert 0.2632409895178194 mit der Ungenauigkeit 0.017053146577525387 berechnet sich das
Ergebnis zu:
 0.263 \pm 0.017
-----------------
```

Out[20]: (Decimal('0.263'), Decimal('0.017'))

Aufgabe 2

In [21]:
```
g_sym, pk_sym, pf_sym, eta_sym, lam_sym, r_sym = sp.symbols('g, p_k, p_f, \\eta, v_\\text{lam}, r
params= [g_sym, pk_sym, pf_sym, eta_sym, lam_sym, r_sym]

v_lam = 2/9 * g_sym * (pk_sym - pf_sym) / (eta_sym * lam_sym) * r_sym**2 /1000
```

```
r_val = df["r [mm]"].to_numpy()

theoretische_geschwindigkeiten = []
for n in range(len(lam_val)):

    values = np.array([[
        g_hd, err_g_hd,
        1385, 15,
        1148.2, 0.5,
        0.263, 0.017,
        float(lam_val[n]), float(lam_err[n]),
        r_val[n], 0.013
    ]])

    result = calc_everything(v_lam, params, values, [], False) # mm/s
    theoretische_geschwindigkeiten.append(result)
```

```
C:\Users\finnz\AppData\Local\Temp\ipykernel_5428\2988192940.py:16: FutureWarning: Calling float o
n a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.i
loc[0]) instead
  float(lam_val[n]), float(lam_err[n]),
0.48392428102050733 \pm 0.05052235525249549
1.9093354900964767 \pm 0.17983905216023535
4.238285017843714 \pm 0.3906600926957551
7.434836681133249 \pm 0.6799769999384858
11.464935069037251 \pm 1.0447416235196376
16.29628409074963 \pm 1.4820389857941325
21.89823481492252 \pm 1.9890973782280608
28.241681781570904 \pm 2.5632830298524834
35.298967049145965 \pm 3.2020925464702312
```

In [22]:

```
print(theoretische_geschwindigkeiten[4][0][1])

for n in range(len(lam_val)):
    try:
        print(round_sig_digs(theoretische_geschwindigkeiten[n][0][1], theoretische_geschwindigkei
    except IndexError:
        print("No")
```

```
1.0447416235196376
Aus den Messwert 0.48392428102050733 mit der Ungenauigkeit 0.05052235525249549 berechnet sich das
Ergebnis zu:
 0.48 \pm 0.05
-----------------

(Decimal('0.48'), Decimal('0.05'))
Aus den Messwert 1.9093354900964767 mit der Ungenauigkeit 0.17983905216023535 berechnet sich das
Ergebnis zu:
 1.91 \pm 0.18
-----------------

(Decimal('1.91'), Decimal('0.18'))
Aus den Messwert 4.238285017843714 mit der Ungenauigkeit 0.3906600926957551 berechnet sich das Er
gebnis zu:
 4.2 \pm 0.4
-----------------

(Decimal('4.2'), Decimal('0.4'))
Aus den Messwert 7.434836681133249 mit der Ungenauigkeit 0.6799769999384858 berechnet sich das Er
gebnis zu:
 7.4 \pm 0.7
-----------------

(Decimal('7.4'), Decimal('0.7'))
No
Aus den Messwert 16.29628409074963 mit der Ungenauigkeit 1.4820389857941325 berechnet sich das Er
gebnis zu:
 16 \pm 1.5
-----------------

('16', Decimal('1.5'))
Aus den Messwert 21.89823481492252 mit der Ungenauigkeit 1.9890973782280608 berechnet sich das Er
gebnis zu:
 21 \pm 1.10
-----------------

('21', Decimal('1.10'))
Aus den Messwert 28.241681781570904 mit der Ungenauigkeit 2.5632830298524834 berechnet sich das E
rgebnis zu:
 28 \pm 2.6
-----------------

('28', Decimal('2.6'))
Aus den Messwert 35.298967049145965 mit der Ungenauigkeit 3.2020925464702312 berechnet sich das E
rgebnis zu:
 35 \pm 3
-----------------

('35', Decimal('3'))
```

Reynolds

In [23]:
```python
rho_sym, v_sym, L_sym, eta_sym = sp.symbols('\\rho_f, v, L, \\eta')
params= [rho_sym, v_sym, L_sym, eta_sym]

re = rho_sym * v_sym * L_sym / eta_sym

# kg/m^3
rho_val = 1148.2
rho_err = 0.5
```

```python
# m
L_val = (df["d [mm]"]) /1000
L_err = (df["err d"]) /1000

print(L_val[0])

# kg / (m s)
eta_val = 0.263
eta_err = 0.017

# m/s

reynolds_zahlen = []
for n in range(len(lam_val)):
    values = np.array([[
        rho_val, rho_err,
        float(v_val[n]), float(v_err[n]),
        float(L_val[n]), float(L_err[n]),
        eta_val, eta_err,
    ]])

    result = calc_everything(re, params, values, [], False)
    reynolds_zahlen.append(result / 1000)

print(reynolds_zahlen)
```

```
0.001
2.761753205769616 \pm 0.22124673840248085
C:\Users\finnz\AppData\Local\Temp\ipykernel_5428\3383050494.py:26: FutureWarning: Calling float o
n a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.i
loc[0]) instead
  float(v_val[n]), float(v_err[n]),
17.477099550362723 \pm 1.3626358459836931
59.53335637746285 \pm 4.139069944941397
130.58163911307904 \pm 8.8746131982553
253.8243876558493 \pm 17.807499574597585
393.31346555681165 \pm 27.258033673492573
627.5247694818122 \pm 45.42433146859816
884.2084997834144 \pm 66.93291396981454
1212.716518800169 \pm 97.2400921302415
[array([[0.00276175, 0.00022125]]), array([[0.0174771 , 0.00136264]]), array([[0.05953336, 0.0041
3907]]), array([[0.13058164, 0.00887461]]), array([[0.25382439, 0.0178075 ]]), array([[0.3933134
7, 0.02725803]]), array([[0.62752477, 0.04542433]]), array([[0.8842085 , 0.06693291]]), array
([[1.21271652, 0.09724009]])]
```

In [24]:

```python
for n in range(len(lam_val)):
    try:
        print(round_sig_digs(reynolds_zahlen[n][0][1], reynolds_zahlen[n][0][0]))
    except IndexError:
        print("No")
```

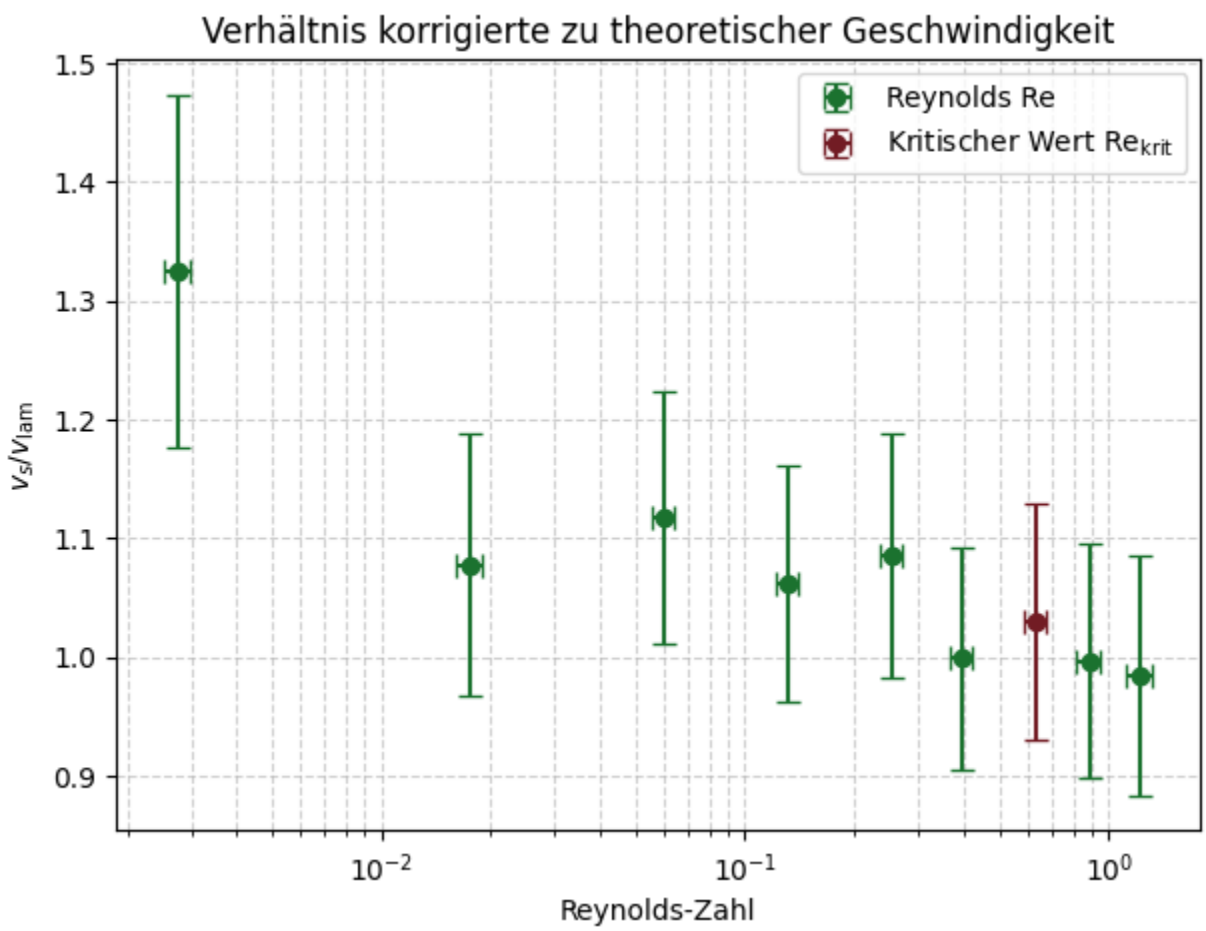
```
Aus den Messwert 0.0027617532057696162 mit der Ungenauigkeit 0.00022124673840248085 berechnet sic
h das Ergebnis zu:
 0.00276 \pm 0.00022
-----------------

(Decimal('0.00276'), Decimal('0.00022'))
Aus den Messwert 0.017477099550362724 mit der Ungenauigkeit 0.001362635845983693 berechnet sich d
as Ergebnis zu:
 0.0175 \pm 0.0014
-----------------

(Decimal('0.0175'), Decimal('0.0014'))
Aus den Messwert 0.05953335637746285 mit der Ungenauigkeit 0.004139069944941397 berechnet sich da
s Ergebnis zu:
 0.060 \pm 0.004
-----------------

(Decimal('0.060'), Decimal('0.004'))
Aus den Messwert 0.13058163911307905 mit der Ungenauigkeit 0.008874613198255301 berechnet sich da
s Ergebnis zu:
 0.131 \pm 0.009
-----------------

(Decimal('0.131'), Decimal('0.009'))
Aus den Messwert 0.2538243876558493 mit der Ungenauigkeit 0.017807499574597586 berechnet sich das
Ergebnis zu:
 0.254 \pm 0.018
-----------------

(Decimal('0.254'), Decimal('0.018'))
Aus den Messwert 0.3933134655568116 mit der Ungenauigkeit 0.02725803367349257 berechnet sich das
Ergebnis zu:
 0.393 \pm 0.027
-----------------

(Decimal('0.393'), Decimal('0.027'))
Aus den Messwert 0.6275247694818122 mit der Ungenauigkeit 0.04542433146859816 berechnet sich das
Ergebnis zu:
 0.63 \pm 0.05
-----------------

(Decimal('0.63'), Decimal('0.05'))
Aus den Messwert 0.8842084997834143 mit der Ungenauigkeit 0.06693291396981453 berechnet sich das
Ergebnis zu:
 0.88 \pm 0.07
-----------------

(Decimal('0.88'), Decimal('0.07'))
Aus den Messwert 1.2127165188001692 mit der Ungenauigkeit 0.0972400921302415 berechnet sich das E
rgebnis zu:
 1.2 \pm 0.10
-----------------

(Decimal('1.2'), Decimal('0.10'))
```

Lograitmischer Plot von v/v_lam über Re

In [25]:
```
# theoretische Geschwindigkeiten (Wert, Fehler)
v_lam_raw = [
np.array([[0.48392428, 0.05052236]]),
np.array([[1.90933549, 0.17983905]]),
```

```
np.array([[4.23828502, 0.39066009]]),
np.array([[7.43483668, 0.679977]]),
np.array([[11.46493507, 1.04474162]]),
np.array([[16.29628409, 1.48203899]]),
np.array([[21.89823481, 1.98909738]]),
np.array([[28.24168178, 2.56328303]]),
np.array([[35.29896705, 3.20209255]])
]

# Reynolds-Zahlen (Wert, Fehler)
re_raw = [
np.array([[0.00276175, 0.00022125]]),
np.array([[0.0174771, 0.00136264]]),
np.array([[0.05953336, 0.00413907]]),
np.array([[0.13058164, 0.00887461]]),
np.array([[0.25382439, 0.0178075]]),
np.array([[0.39331347, 0.02725803]]),
np.array([[0.62752477, 0.04542433]]),
np.array([[0.8842085, 0.06693291]]),
np.array([[1.21271652, 0.09724009]])
]

# Messwerte
vs_val = np.array([
0.641447, 2.057646, 4.736364, 7.896311, 12.44186,
16.276276, 22.546201, 28.151899, 34.753086
])

vs_err = np.array([
0.025777, 0.085947, 0.114695, 0.158296, 0.333627,
0.40123, 0.730266, 1.105647, 1.646186
])


v_lam = np.array([v[0,0] for v in v_lam_raw])
v_lam_err = np.array([v[0,1] for v in v_lam_raw])

re = np.array([r[0,0] for r in re_raw])
re_err = np.array([r[0,1] for r in re_raw])


# Verhältnis
ratio = vs_val / v_lam

# Fehlerfortpflanzung
ratio_err = ratio * np.sqrt(
    (vs_err / vs_val)**2 +
    (v_lam_err / v_lam)**2
)


plt.figure(figsize=(7,5))

# Kritischer Knick
i = 6

mask = np.arange(len(re)) != i

plt.errorbar(
    re[mask], ratio[mask],
    xerr=re_err[mask],
    yerr=ratio_err[mask],
```

```
    fmt='o',
    capsize=4,
    color=gruen,
    label="Reynolds Re"
)

# kritischer Wert
plt.errorbar(
    re[i], ratio[i],
    xerr=re_err[i],
    yerr=ratio_err[i],
    fmt='o',
    capsize=4,
    color=weinrot,
    label=r"Kritischer Wert Re$_\text{krit}$"
)

plt.xscale("log")

plt.xlabel("Reynolds-Zahl")
plt.ylabel(r"$v_s / v_\text{lam}$")
plt.title("Verhältnis korrigierte zu theoretischer Geschwindigkeit")
plt.grid(True, which="both", linestyle="--", alpha=0.6)

plt.legend()
plt.savefig("../img/plot/ver_ges_gegen_re.pdf", dpi=300)
plt.show()
```



Aufgabe 3

In [56]:
```
# cm^3
vol = np.array([5, 10, 15, 20, 25, 30]) /1e6
```

```
vol_err = 0.10 /1e6

# s
t = np.array([92.4, 233.8, 382.8, 540.9, 688.4, 841.9])
t_err = 1.0

# mm
h_a = 531 / 1000
h_e = 524 / 1000
h_err = 3 / 1000

# mm
d_kap = 1.5  / 1000
r_kap = d_kap / 2  / 1000
d_kap_err = 0.01  / 1000
r_kap_err = d_kap_err / 2  / 1000

l_kap = 100.0  / 1000
l_kap_err = 0.5  / 1000
```

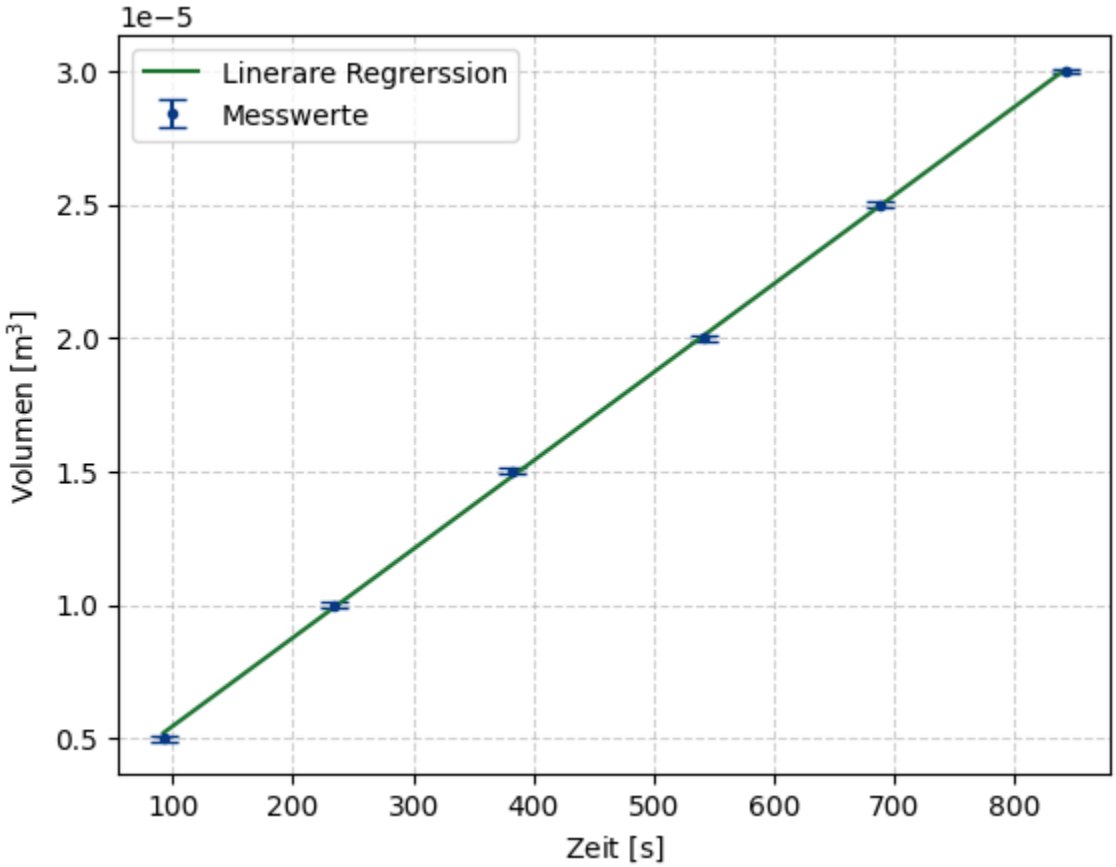
Volumenfluss bestimmen:

In [61]:

```
y = vol
y_err = y * np.sqrt((vol_err/vol)**2 + (t_err/t)**2)

# lineare regression
def linear(x, a, b):
    return a*x + b

popt, pcov = curve_fit(linear, t, y, sigma=y_err, absolute_sigma=True)

slope = popt[0]
intercept = popt[1]

slope_err = np.sqrt(pcov[0,0])
intercept_err = np.sqrt(pcov[1,1])

# smooth fit line
t_fit = np.linspace(min(t), max(t), 200)
y_fit = linear(t_fit, slope, intercept)

# plot
plt.errorbar(
    t,
    y,
    yerr=y_err,
    fmt='.',
    label="Messwerte",
    capsize=5,
    color=blau)


plt.plot(
    t_fit,
    y_fit,
    label="Linerare Regrerssion",
    color= gruen)

plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.xlabel("Zeit [$\mathrm{s}$]")
plt.ylabel(r'Volumen [$\mathrm{m^3}$]')
```

```
plt.legend()
plt.savefig("../img/plot/volumenfluss.pdf", dpi=300)
plt.show()

# print results
print(f"Steigung = {slope:.6e} \\pm {slope_err:.6e}")
print(f"y-Achsenabschnitt = {intercept:.6e} \\pm {intercept_err:.6e}")
```

```
<>:38: SyntaxWarning: invalid escape sequence '\m'
<>:38: SyntaxWarning: invalid escape sequence '\m'
C:\Users\finnz\AppData\Local\Temp\ipykernel_5428\2296362686.py:38: SyntaxWarning: invalid escape
sequence '\m'
  plt.xlabel("Zeit [$\mathrm{s}$]")
```




```
Steigung = 3.318474e-08 \pm 1.732220e-10
y-Achsenabschnitt = 2.125926e-06 \pm 9.313616e-08
```

In [58]:

```
rhof_sym, g_sym, ha_sym, he_sym = sp.symbols(r'\rho_f, g, h_\text{A}, h_\text{E}')
params = [rhof_sym, g_sym, ha_sym, he_sym]

p_diff = rhof_sym * g_sym / 2 * (ha_sym + he_sym)

pf_val = 1148.2
pf_err = 0.5

values = np.array([
    [
    pf_val, pf_err,
    g_hd, err_g_hd,
    h_a, h_err,
    h_e, h_err
    ]
])

print(values)
```

```
calc_everything(p_diff, params, values, [], False)
```

```
[[1.14820e+03 5.00000e-01 9.80984e+00 2.00000e-05 5.31000e-01 3.00000e-03
  5.24000e-01 3.00000e-03]]
5941.5797469200015 \pm 24.033507725013767
```

Out[58]: array([[5941.57974692, 24.03350773]])

Eta bestimmen

In [59]:
```
p_sym, R_sym, L_sym, Vt_sym = sp.symbols('p, R, L, \\dot{V}')
params = [p_sym, R_sym, L_sym, Vt_sym]

# Pa
rho_val = 594
rho_err = 24

# m^3/s * 10^{-8}
vt_val = 3.318
vt_err = 0.017

values = np.array([
    [
    rho_val, rho_err,
    r_kap, r_kap_err,
    l_kap, l_kap_err,
    vt_val, vt_err,
    ]
])

eta = (sp.pi * p_sym * R_sym**4) / (8 * L_sym * Vt_sym)

calc_everything(eta, params, values, [], True)
```

```
2.224410838707949e-22 \pm 1.0885641540387825e-23
gegebene Funktion:
Die gegebene Funktion lautet:
```

$$\frac{\pi R^4 p}{8 L \dot{V}}$$

```
Hier ist der dazugehörige Latex code:
```

```
\frac{\pi R^{4} p}{8 L \dot{V}}
```

Kopieren

```
Formel des absoluten Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:
```

$$\frac{\pi}{8}\sqrt{\frac{R^6}{L^4\dot{V}^4}\left(L^2R^2p^2\Delta\dot{V}^2 + L^2\dot{V}^2\left(R^2\Delta p^2 + 16p^2\Delta R^2\right) + R^2\dot{V}^2p^2\Delta L^2\right)}$$

```
Hier ist der dazugehörige Latex code:
```

```
\frac{\pi}{8} \sqrt{\frac{R^{6}}{L^{4} \dot{V}^{4}} \left(L^{2} R^{2} p^{2} \Delta \dot{V}^{2} + L^{2} \dot{V}^{2} \left(R^{2} \Delta p^{2} + 16 p^{2} \Delta R^{2}\right) + R^{2} \dot{V}^{2} p^{2} \Delta L^{2}\right)}
```

Kopieren

Formel des relativen Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:

$$\sqrt{\frac{\Delta p^{2}}{p^{2}} + \frac{\Delta \dot{V}^{2}}{\dot{V}^{2}} + \frac{16 \Delta R^{2}}{R^{2}} + \frac{\Delta L^{2}}{L^{2}}}$$

Hier ist der dazugehörige Latex code:

```
\sqrt{\frac{\Delta p^{2}}{p^{2}} + \frac{\Delta \dot{V}^{2}}{\dot{V}^{2}} + \frac{16 \Delta R^{2}}{R^{2}} + \frac{\Delta L^{2}}{L^{2}}}
```

Kopieren

Out[59]: array([[2.22441084e-22, 1.08856415e-23]])

Reynolds-Zahl

In [60]:
```python
Vt, rho_f, L, R = sp.symbols(r'\dot{V}, \rho_f, L, R')
params = [Vt, rho_f, L, R]

values = np.array([
    [
    vt_val, vt_err,
    pf_val, pf_err,
    l_kap, l_kap_err,
    d_kap, d_kap_err,
    ]
])

Re_hp = 2 * (Vt * rho_f) / (L * R * sp.pi)

calc_everything(Re_hp, params, values, [])
```

16168986.116629949 \pm 158328.16746938726
gegebene Funktion:
Die gegebene Funktion lautet:

$$\frac{2\dot{V}\rho_f}{\pi L R}$$

Hier ist der dazugehörige Latex code:

```
\frac{2 \dot{V} \rho_{f}}{\pi L R}
```

Kopieren

Formel des absoluten Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:

$$\frac{2}{\pi}\sqrt{\frac{1}{L^4R^4}\left(L^2R^2\left(\dot{V}^2\Delta\rho_f^2+\rho_f^2\Delta\dot{V}^2\right)+L^2\dot{V}^2\rho_f^2\Delta R^2+R^2\dot{V}^2\rho_f^2\Delta L^2\right)}$$

Hier ist der dazugehörige Latex code:

```
\frac{2}{\pi} \sqrt{\frac{1}{L^{4} R^{4}} \left(L^{2} R^{2} \left(\dot{V}^{2} \Delta \rho_{f}^{2} + \rho_{f}^{2} \Delta \dot{V}^{2}\right) + L^{2} \dot{V}^{2} \rho_{f}^{2} \Delta R^{2} + R^{2} \dot{V}^{2} \rho_{f}^{2} \Delta L^{2}\right)}
```

Kopieren

Formel des relativen Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:

$$\sqrt{\frac{\Delta\rho_f^2}{\rho_f^2}+\frac{\Delta\dot{V}^2}{\dot{V}^2}+\frac{\Delta R^2}{R^2}+\frac{\Delta L^2}{L^2}}$$

Hier ist der dazugehörige Latex code:

```
\sqrt{\frac{\Delta \rho_{f}^{2}}{\rho_{f}^{2}} + \frac{\Delta \dot{V}^{2}}{\dot{V}^{2}} + \frac{\Delta R^{2}}{R^{2}} + \frac{\Delta L^{2}}{L^{2}}}
```

Kopieren

Out[60]: array([[16168986.11662995, 158328.16746939]])

Signifikante Abweichungen der Geschwindigkeiten

In [ ]:

```
# v_gem
v_gem = np.array([0.63, 2.00, 4.55, 7.48, 11.63, 15.02, 20.53, 25.3, 30.9])
v_gem_err = np.array([0.03, 0.08, 0.07, 0.07, 0.23, 0.20, 0.18, 0.4, 0.3])

# v_korr
v_korr = np.array([0.641, 2.06, 4.74, 7.90, 12.4, 16.3, 22.5, 28.15, 34.8])
v_korr_err = np.array([0.026, 0.09, 0.11, 0.16, 0.3, 0.4, 0.7, 1.11, 1.6])

# v_theo
v_theo = np.array([0.48, 1.91, 4.2, 7.4, 11.5, 16.3, 21.90, 28.2, 35])
v_theo_err = np.array([0.05, 0.18, 0.4, 0.7, 1.0, 1.5, 1.10, 2.6, 3])

def sigma_abweichung(p1, p2, err_p1=0, err_p2=0):
    if err_p1 == err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert fehlerbehaftet sein!'
    abweichung = abs(p1 - p2) / np.sqrt(err_p1**2 + err_p2**2)
    return abweichung

sigma_list = []

for vg, vg_err, vk, vk_err in zip(v_gem, v_gem_err, v_korr, v_korr_err):
    sigma = sigma_abweichung(vg, vk, vg_err, vk_err)
    sigma_list.append(f"{sigma:.2f}")  # rundet auf 2 Nachkommastellen und hängt \sigma an

# In ein numpy array umwandeln, wenn nötig
sigma_array = np.array(sigma_list)

print(sigma_array)
```

```
['0.28' '0.50' '1.46' '2.40' '2.04' '2.86' '2.73' '2.42' '2.40']
```

In [104…

```python
sigma_list = []

for vt, vt_err, vk, vk_err in zip(v_theo, v_theo_err, v_korr, v_korr_err):
    sigma = sigma_abweichung(vt, vk, vt_err, vk_err)
    sigma_list.append(f"{sigma:.2f}")  # rundet auf 2 Nachkommastellen und hängt \sigma an

# In ein numpy array umwandeln, wenn nötig
sigma_array = np.array(sigma_list)

print(sigma_array)
```

```
['2.86' '0.75' '1.30' '0.70' '0.86' '0.00' '0.46' '0.02' '0.06']
```